



Roteiro de Aprendizagem Streamlit + Pandas para iniciantes

Guia didático para construção do dashboard do setor Atende

Público-alvo: alunos sem conhecimento prévio de Python, Pandas ou Streamlit

Versão 1.0 — Abril de 2026

Universidade Católica de Brasília (UCB / UBEC)

Sumario

Use este indice para navegar pelo material no seu ritmo.

Como usar este roteiro	pag. 4
Parte 1 — Boas-vindas e visao do projeto	pag. 5
Parte 2 — Fundamentos de logica de programacao	pag. 7
Parte 3 — Pandas: tabelas inteligentes	pag. 16
Parte 4 — Streamlit: do zero a primeira tela	pag. 28
Parte 5 — Checklist por tela do Figma	pag. 38
Parte 6 — Pandas + Streamlit juntos	pag. 46
Parte 7 — Extra: SQL no Pandas com DuckDB	pag. 49
Parte 8 — Recursos, glossario e proximos passos	pag. 53

As paginas sao aproximadas e podem variar de acordo com a leitura.

Como usar este roteiro

Antes de começar, leia esta página com atenção.

Este roteiro foi escrito para quem nunca programou. A ideia é que você avance no seu ritmo, sem pular as analogias e sem ignorar as perguntas-guia. As respostas não estão no documento — você vai encontrá-las pesquisando, experimentando e errando no próprio computador. Isso é proposital: programar é, antes de tudo, aprender a investigar.

Ao longo do material você vai encontrar quatro tipos de caixas destacadas. Cada uma tem um propósito diferente:

[CONCEITO]

CONCEITO — apresenta uma ideia nova, normalmente com uma analogia do dia a dia para facilitar o entendimento. Leia com calma e tente repetir a definição com suas próprias palavras.

[DICA]

DICA — sugere uma boa prática, um atalho de teclado ou um comportamento que evita erros comuns. São pequenos truques que economizam tempo.

[PERGUNTA]

PERGUNTA — desafios para você pensar e pesquisar. Não tem resposta no roteiro de propósito. Use a documentação oficial, o Google e ferramentas de IA (ChatGPT, Claude, Gemini) como apoio.

[LEITURA]

LEITURA — links para documentação oficial, tutoriais ou vídeos confiáveis. Sempre que ver uma referência dessas, abra antes de seguir adiante.

Uma sugestão final: tenha sempre o computador ao lado enquanto lê. Ler sobre programação sem digitar código não funciona. Você precisa testar, errar e ver as mensagens de erro para o conhecimento se fixar.

Parte 1 — Boas-vindas e visao do projeto

Antes de aprender qualquer linha de codigo, e essencial entender o que voce esta construindo e por que.

O que e o setor Atende

O Atende e o ponto principal de contato entre o aluno e a Universidade Catolica de Brasilia. Toda vez que um estudante precisa trocar de curso, ajustar horarios, aproveitar disciplinas de outra faculdade ou tirar duvidas sobre mensalidade, e ao Atende que ele recorre. O setor opera em tres frentes: atendimento presencial (com retirada de senha), call center remoto e back-office (analistas que resolvem as demandas internas).

Qual o problema?

O volume de solicitacoes ultrapassa a capacidade da equipe. Muitas duvidas sao simples e poderiam ser resolvidas no Portal do Aluno — mas o portal nao e intuitivo o bastante. Outras demandas dependem de setores externos (Secretaria Academica, UBEC) que tem prazos proprios, e o aluno, sem saber o status, volta repetidamente ao Atende. Resultado: senhas se acumulam, atendentes ficam sobrecarregados, e processos simples viram gargalos.

O que voces vao construir

Um **dashboard web em Streamlit** (uma especie de site feito com Python) que le os dados das solicitacoes a partir de planilhas (.csv, .xlsx ou .json) e mostra tudo de forma organizada, filtravel e visualmente clara. Quatro telas iniciais ja foram desenhadas no Figma:

• **Login Portal-Atende** — porta de entrada do dashboard.

• **Calendario de Solicitacoes** — visualizacao mensal das demandas, com cores por urgencia e por setor.

• **Lista de Solicitacoes** — listagem de todas as solicitacoes com filtros por urgencia, status e periodo.

• **Lista de Retornos** — respostas dos setores ao aluno, com filtros por setor e estado de leitura.

[CONCEITO]

Dashboard e uma palavra inglesa que significa *painel de controle*. Pense no painel do carro: voce nao mexe no motor, mas ve a velocidade, o combustivel, a temperatura. Um dashboard de software faz a mesma coisa com dados — voce ve o que importa de relance e toma decisoes.

Linha do tempo sugerida

Semana	Foco	Entregavel
1	Logica de programacao + instalacoes (Parte 2)	Python rodando, primeiro 'Hello World'
2	Pandas basico (Parte 3, ate filtros)	Ler um .csv e mostrar as primeiras linhas

3	Pandas avancado (Parte 3, restante)	Agrupamentos e datas funcionando
4	Streamlit basico (Parte 4)	App com sidebar e tres componentes
5	Tela de Login + Lista de Solicitacoes (Parte 5)	Duas telas conectadas
6	Calendario + Lista de Retornos (Parte 5)	As 4 telas integradas
7	Pandas + Streamlit juntos (Parte 6)	Filtros funcionando ponta a ponta
8	DuckDB e revisao (Parte 7) + apresentacao	Versao final pronta para entrega

[DICA]

Esta linha do tempo e uma sugestao, nao uma regra. Se voce travar em alguma parte, volte e refaca os exercicios da secao anterior antes de seguir.

[PERGUNTA]

Antes de continuar, escreva no caderno (ou num arquivo .txt) com suas proprias palavras: **qual e o problema do Atende?** e **como o dashboard ajuda a resolve-lo?** Isso vai te ajudar quando voce explicar o projeto na apresentacao final.

Parte 2 — Fundamentos de logica de programacao

Antes de mergulhar em Pandas e Streamlit, voce precisa entender como o Python pensa. Esta parte e a mais lenta de proposito — pule-a com cuidado.

2.1. O que e um programa

[CONCEITO]

Um **programa** e uma sequencia de instrucoes que o computador executa de cima para baixo. Pense em uma *receita de bolo*: voce lista os ingredientes, a ordem dos passos e, no final, tem um bolo. Se voce trocar a ordem dos passos ou esquecer um ingrediente, o resultado muda — ou nem sai. Programacao e a mesma coisa, so que feita com texto.

Quando voce roda um programa em Python, o interpretador le seu codigo linha por linha, decide o que fazer e mostra o resultado (na tela, num arquivo ou onde voce mandar). Ate algo simples como mostrar a frase "Bem-vindo ao Atende" e um programa.

2.2. Instalando o Python e o VS Code

Python e a linguagem de programacao que voces vao usar. **VS Code** (Visual Studio Code) e um editor de texto avancado, gratuito, feito pela Microsoft. Pense no VS Code como o Word, mas para escrever codigo em vez de cartas. Os passos abaixo sao um resumo — siga as instrucoes oficiais para a sua versao do Windows/Mac.

bullet Baixe o Python em python.org/downloads. Marque a opcao "Add Python to PATH" antes de instalar.

bullet Baixe o VS Code em code.visualstudio.com.

bullet Abra o VS Code e instale a extensao oficial chamada **Python** (icone azul, da Microsoft).

bullet Abra o terminal do VS Code (atalho Ctrl + acento grave) e digite `python --version`. Se aparecer um numero (ex: 3.12.0), deu certo.

[DICA]

Sempre que voce instalar uma biblioteca nova, use o terminal: `pip install nome-da-biblioteca`. O **pip** e um "gerenciador de pacotes" — algo como uma loja gratuita de bibliotecas Python.

[LEITURA]

Doc oficial Python: <https://docs.python.org/pt-br/3/tutorial/index.html>
Curso em Video (Gustavo Guanabara, em portugues):
<https://www.cursoemvideo.com/curso/python-3-mundo-1/>

2.3. Variaveis: caixas com etiqueta

[CONCEITO]

Uma **variavel** e uma caixa onde voce guarda um valor e cola uma etiqueta com um nome. Depois, quando voce quiser usar o valor, voce nao precisa lembrar do conteudo — basta chamar pela etiqueta. **Exemplo:** `nome_aluno = "Joao Salles"` cria uma caixa chamada `nome_aluno` com o texto "Joao Salles" dentro.

Em Python voce nao precisa avisar o tipo da variavel. Basta escrever `idade = 21` e o Python entende sozinho que e um numero. Se voce escrever `idade = "21"` (com aspas), ele entende como texto. Aspas mudam tudo!

[DICA]

Use nomes de variaveis em **portugues, descritivos e em snake_case** (palavras separadas por underline). `total_solicitacoes` e melhor que `x`. O codigo e lido muito mais vezes do que escrito.

[PERGUNTA]

Crie no VS Code um arquivo teste.py, declare tres variaveis (seu nome, sua idade e seu curso) e mostre as tres na tela usando `print(...)`. Que sintaxe voce precisa aprender para misturar texto com variaveis dentro do print?

2.4. Tipos de dados

Tipo	Como e em Python	Exemplo	Para que serve
Texto	<code>str (string)</code>	<code>"Solicitacao 32"</code>	nomes, descricoes, frases
Numero inteiro	<code>int</code>	<code>42</code>	contagem, idade, quantidade
Numero decimal	<code>float</code>	<code>31.50</code>	precos, medias, percentuais
Verdadeiro/falso	<code>bool</code>	<code>True / False</code>	condicoes, flags, sim/nao
Lista (sequencia)	<code>list</code>	<code>["A", "B", "C"]</code>	varias coisas em ordem
Dicionario (mapa)	<code>dict</code>	<code>{"nome": "Ana", "idade": 22}</code>	uma chave-valor (como uma agenda)

[CONCEITO]

Lista e uma fila de coisas em ordem (como a fila de senhas do Atende). Voce acessa pelo numero da posicao: `solicitacoes[0]` pega a primeira. **Dicionario** e uma agenda: voce procura por uma chave (palavra) e recebe o valor associado. `aluno["nome"]` devolve o nome do aluno.

[PERGUNTA]

Pesquise: o que acontece se voce somar um `int` com uma `str` em Python (ex: `3 + "3"`)? Que erro aparece? Que mensagem o Python te da? Aprender a ler erros e metade do trabalho de programar.

2.5. Operadores

Aritmeticos: + - * / (somar, subtrair, multiplicar, dividir), // (divisao inteira), % (resto da divisao), ** (potencia).

Comparacao: == (igual a), != (diferente de), < > <= >=. **Atencao:** = atribui valor; == compara. Confundir os dois e o erro mais comum.

Logicos: and (E), or (OU), not (NAO). Servem para combinar varias condicoes. Exemplo: idade >= 18 and curso == "Direito".

[DICA]

Decore: **uma igualdade compara, atribuicao usa um sinal so**. Se o Python reclamar de "SyntaxError" perto de um =, e quase certo que voce devia ter usado ==.

2.6. Indentacao: o espaco que importa

[CONCEITO]

Em Python, o espaco em branco no comeco da linha (a **indentacao**) decide quem pertence a quem. Outras linguagens usam chaves { }; o Python usa indentacao. A regra geral: **4 espacos por nivel**. O VS Code faz isso pra voce automaticamente quando voce aperta Tab apos um ":".

Exemplo:

```
if urgencia == "maxima":
    print("Atender primeiro!")
    avisar_supervisor()
```

As duas linhas com 4 espacos de recuo pertencem ao if. Se voce remover os espacos, o Python reclama com um "IndentationError".

2.7. Condicionais: tomando decisoes

[CONCEITO]

Um **condicional** e como um porteiro decidindo quem entra. Se a condicao for verdadeira (True), o codigo de dentro roda. Se for falsa, ele e ignorado. Em Python: if (se), elif (senao se), else (senao).

```
if status == "pendente":
    print("Ainda nao analisado")
elif status == "em andamento":
    print("Em analise")
else:
    print("Concluido ou cancelado")
```


[PERGUNTA]

Escreva um pequeno programa que recebe a urgencia de uma solicitacao ("maxima", "media" ou "minima") e mostra na tela quantos dias o aluno deve esperar. Voce **nao** sabe ainda o que e `input(...)` — pesquise.

2.8. Repeticoes (loops)

[CONCEITO]

Um **loop** e uma repeticao automatica. Imagine que voce precisa carimbar 200 documentos: voce nao escreve a mesma instrucao 200 vezes — usa um loop. Em Python ha dois principais: `for` ("para cada item da lista, faca isso") e `while` ("enquanto a condicao for verdadeira, repita").

```
for solicitacao in lista_de_solicitacoes:
    print(solicitacao)
```

[DICA]

Quase sempre voce vai usar `for`. Use `while` apenas quando voce **nao sabe** quantas vezes vai repetir (ex: "continue tentando ate o usuario digitar uma senha valida").

2.9. Funcoes: mini-receitas reutilizaveis

[CONCEITO]

Uma **funcao** e um pedaco de codigo que voce empacota com um nome e pode chamar quando quiser. Pense numa receita de molho de tomate: voce escreve uma vez e usa em pizza, lasanha, espaguete... Em Python, voce define com `def` e chama escrevendo o nome com parenteses.

```
def calcular_dias_atraso(data_limite, hoje):
    diferenca = hoje - data_limite
    return diferenca.days

atraso = calcular_dias_atraso(data_limite, hoje)
```

Os valores entre parenteses na chamada da funcao sao os **argumentos**. O `return` e o que a funcao devolve para quem a chamou. Toda vez que voce sentir que escreveu o mesmo codigo duas vezes, transforme em funcao.

2.10. Importando bibliotecas

[CONCEITO]

Uma **biblioteca** é um conjunto de funções prontas que outras pessoas escreveram para você usar. Em vez de reinventar a roda, você importa. Para o nosso projeto, as duas estrelas são **pandas** (manipular tabelas) e **streamlit** (criar interfaces web). Você importa com a palavra-chave `import`.

```
import pandas as pd
import streamlit as st
```

as `pd` cria um apelido curto. Em vez de digitar `pandas.read_csv(...)` você digita `pd.read_csv(...)`. **pd** e **st** são convenções: a comunidade inteira escreve assim. Mantenha o padrão.

[PERGUNTA]

Pesquise no Google: qual a diferença entre `import pandas`, `import pandas as pd` e `from pandas import DataFrame`? Quando cada forma é mais útil?

2.11. Erros: aliados, não inimigos

Erros são parte do processo. O Python escreve **tracebacks** (relatórios de erro) que parecem assustadores, mas são na verdade um mapa: a última linha diz o tipo do erro, e as linhas anteriores mostram onde ele aconteceu. Aprenda a ler de baixo para cima.

[DICA]

Quando ver um erro, copie a mensagem inteira e cole no Google. 99% das dúvidas de iniciante já foram respondidas no Stack Overflow. Também peça ajuda ao ChatGPT/Claude/Gemini explicando o que você já tentou.

[LEITURA]

Documentação oficial Python (em português): <https://docs.python.org/pt-br/3/>
W3Schools Python: <https://www.w3schools.com/python/>
Curso em Vídeo — Mundo 1: <https://www.cursoemvideo.com/curso/python-3-mundo-1/>

[PERGUNTA]

Antes de avançar para a Parte 3, faça este teste: explique para alguém (ou para você mesmo, em voz alta) **o que são variáveis, condicionais, loops e funções**. Se travou em algum, releia a seção correspondente. Sem esses quatro fundamentos, Pandas e Streamlit não fazem sentido.

Parte 3 — Pandas: tabelas inteligentes

Pandas é a biblioteca que transforma planilhas em estruturas que o Python sabe manipular. Aqui você aprende a ler, filtrar, ordenar e agrupar dados — o coração do dashboard.

3.1. O que é Pandas e por que existe

[CONCEITO]

Pandas é uma biblioteca Python para trabalhar com dados em formato de tabela. Pense em um **Excel turbinado**: você lê planilhas, faz contas, filtra linhas e cria gráficos — tudo escrevendo poucas linhas de código. A grande vantagem sobre o Excel é que tudo fica **automatizado**: você escreve o passo a passo uma vez e ele se repete sem erro humano.

Para o projeto Atende, Pandas vai ler a planilha de solicitações (em .csv, .xlsx ou .json), filtrar pela urgência que o usuário escolheu, agrupar por setor e calcular métricas como "quantas solicitações em andamento". Tudo isso, depois, alimenta o Streamlit.

[LEITURA]

Documentação oficial (versão 3.0.2, março de 2026): [https://pandas.pydata.org/docs/10 minutos com Pandas \(overview rápido, em inglês\): https://pandas.pydata.org/docs/user_guide/10min.html](https://pandas.pydata.org/docs/10%20minutos%20com%20Pandas%20(overview%20rapido,%20em%20ingles):https://pandas.pydata.org/docs/user_guide/10min.html)

3.2. Instalando o Pandas

No terminal do VS Code, digite:

```
pip install pandas openpyxl
```

O **openpyxl** é uma biblioteca complementar que o Pandas usa por baixo dos panos para ler arquivos .xlsx (Excel). Sem ele, você só consegue ler .csv e .json.

3.3. Series e DataFrame: as duas peças centrais

[CONCEITO]

DataFrame é uma tabela com linhas e colunas, como uma aba do Excel. **Series** é uma coluna isolada (uma fila de valores com um nome). Quando você extrai uma coluna de um DataFrame, você recebe uma Series. Lembre-se: tabela = DataFrame, coluna = Series.

```
import pandas as pd

df = pd.DataFrame({
    "protocolo": [101, 102, 103],
    "urgencia": ["maxima", "media", "minima"],
    "status": ["em andamento", "pendente", "pendente"]
})
```

[DICA]

A convencao da comunidade e chamar a variavel principal de `df` (de `DataFrame`). Se voce tiver varias tabelas, use nomes descritivos: `df_solicitacoes`, `df_retornos`.

3.4. Lendo arquivos: csv, xlsx e json

Formato	Funcao	Exemplo basico
.csv (texto separado por virgula)	<code>pd.read_csv</code>	<code>pd.read_csv("dados.csv")</code>
.xlsx (Excel)	<code>pd.read_excel</code>	<code>pd.read_excel("dados.xlsx", sheet_name="Solicitacoes")</code>
.json (chave-valor)	<code>pd.read_json</code>	<code>pd.read_json("dados.json")</code>

[DICA]

Tres parametros que salvam vidas: `sep=";"` (quando o CSV usa ponto-e-virgula), `encoding="latin-1"` ou `encoding="utf-8"` (quando aparecem acentos esquisitos), e `sheet_name=...` no `read_excel` (para escolher qual aba).

[PERGUNTA]

Crie um arquivo `solicitacoes.csv` a mao, com 5 linhas inventadas (protocolo, urgencia, status, data). Em seguida, leia esse arquivo com Pandas e mostre o resultado. Que **caminho** voce precisa passar para a funcao? O caminho e relativo (perto do `.py`) ou absoluto (com a pasta inteira)?

3.5. Inspeccionando: head, tail, info, describe

Antes de fazer qualquer analise, voce sempre da uma olhada no que tem na tabela. Os comandos abaixo sao o seu kit de inspecao:

Comando	O que faz
<code>df.head()</code>	Mostra as primeiras 5 linhas (use <code>df.head(10)</code> para 10)
<code>df.tail()</code>	Mostra as ultimas 5 linhas
<code>df.shape</code>	Devolve (linhas, colunas) — ex: (32, 7)
<code>df.columns</code>	Lista os nomes das colunas
<code>df.dtypes</code>	Mostra o tipo de cada coluna
<code>df.info()</code>	Resumo: colunas, tipos, contagem de nao-nulos
<code>df.describe()</code>	Estatisticas (so colunas numericas): media, min, max...

3.6. Seleccionando colunas e linhas

[CONCEITO]

Para pegar uma coluna: `df["urgencia"]`. Para pegar varias: `df[["protocolo", "urgencia"]]` (note as duas chaves: a de fora e selecao, a de dentro e a lista). Para pegar linhas voce usa `.loc[]` (por rotulo) ou `.iloc[]` (por posicao numerica).

```
df["urgencia"]      # uma coluna
df.loc[0]           # linha de rotulo 0
df.iloc[2]          # terceira linha (posicao 2)
df.loc[0, "urgencia"] # linha 0, coluna "urgencia"
```

[DICA]

Quando voce pega uma coluna unica, recebe uma **Series**. Quando passa uma lista, recebe um **DataFrame** (mesmo se a lista tiver so um nome). Esse detalhe ajuda a entender erros futuros.

3.7. Filtros booleanos

[CONCEITO]

Filtrar e a operacao mais comum em Pandas. Voce escreve uma condicao que devolve True/False para cada linha, e o Pandas devolve so as linhas *True*. A logica e a mesma do `if`, mas aplicada em coluna inteira de uma vez.

```
df[df["urgencia"] == "maxima"]
```

Para combinar varias condicoes voce usa `&` (E) e `|` (OU). Cuidado: **cada condicao precisa estar entre parenteses**.

```
df[(df["urgencia"] == "maxima") & (df["status"] == "pendente")]
```

[PERGUNTA]

Como voce filtraria so as solicitacoes com urgencia **media OU minima**? E so as solicitacoes feitas **nos ultimos 7 dias**? (a segunda exige saber lidar com datas — adiantar para a 3.10 vale a pena.)

3.8. Ordenando e contando

`df.sort_values("data_abertura", ascending=False)` — ordena pela coluna data, do mais novo para o mais velho. `ascending=True` e o padrao (crescente).

Para contagens rapidas: `df["urgencia"].value_counts()` devolve quantas vezes cada valor aparece — perfeito para responder "quantas solicitacoes de cada urgencia temos?".

[DICA]

Ordene **antes** de mostrar na tela e voce evita 90% das reclamacoes do usuario sobre "ta tudo embaralhado".

3.9. Agrupamento (groupby): a peça-chave do dashboard

[CONCEITO]

groupby agrupa linhas por uma coluna (ou várias) e aplica uma operação em cada grupo. Pense em uma sala de aula dividindo em equipes por curso: cada equipe pode somar suas notas, tirar a média... groupby e isso, mas com tabelas.

```
df.groupby("setor")["protocolo"].count()
```

Le-se: "agrupe por setor, conte quantos protocolos cada setor tem". O resultado é uma Series com setores no rótulo e contagens nos valores.

Para várias operações ao mesmo tempo, use `.agg(...)`:

```
df.groupby("setor").agg(
    total=("protocolo", "count"),
    tempo_medio=("dias_aberto", "mean"),
)
```

[PERGUNTA]

Como você descobriria, usando groupby, **quantas solicitações de cada urgência existem em cada status?** (Dica: groupby aceita uma lista de colunas como argumento.)

3.10. Trabalhando com datas

[CONCEITO]

Em arquivos .csv as datas chegam como texto ("22/04/2026"). Para fazer contas com elas ("quantos dias faltam para vencer?"), você precisa converter para o tipo **datetime** com `pd.to_datetime`. Depois disso, o acessor `.dt` abre um arsenal de funções (`.dt.day`, `.dt.month`, `.dt.day_name()`, etc.).

```
df["data_abertura"] = pd.to_datetime(df["data_abertura"], format="%d/%m/%Y")
df["mes"] = df["data_abertura"].dt.month
df["dia_semana"] = df["data_abertura"].dt.day_name()
```

[DICA]

O parâmetro `format="%d/%m/%Y"` diz ao Pandas como ler a data. `%d` é dia, `%m` é mês, `%Y` é ano com 4 dígitos. Se você não passar, o Pandas tenta adivinhar — e às vezes troca dia com mês em datas tipo 03/04/2026. Sempre passe o `format`.

3.11. Dados ausentes (NaN)

[CONCEITO]

NaN ("Not a Number") é o valor especial que o Pandas usa quando uma célula está vazia. Você não pode comparar com `==` — tem que usar funções específicas: `.isna()` (é vazio?), `.notna()` (não é vazio?), `.fillna(valor)` (preenche com algo) e `.dropna()` (joga fora as linhas vazias).

```
df["observacao"] = df["observacao"].fillna("sem observacao")
df_limpo = df.dropna(subset=["data_abertura"])
```

3.12. Combinando tabelas: merge e concat

[CONCEITO]

concat empilha tabelas (uma em cima da outra ou lado a lado), como colar pedaços de papel. **merge** conecta tabelas por uma coluna em comum, como um JOIN do SQL — uniforme "junte tudo que tem o mesmo cpf". É vital quando você tem solicitações em uma tabela e os dados do aluno em outra.

```
df_total = pd.concat([df_marco, df_abril])
df_completo = df_solicitacoes.merge(df_alunos, on="matricula", how="left")
```

[LEITURA]

Comparação Pandas com SQL (essencial antes da Parte 7):

https://pandas.pydata.org/docs/getting_started/comparison/comparison_with_sql.html

3.13. Aplicando funções (apply, map)

Quando você precisa transformar uma coluna inteira com uma lógica que não é um operador padrão, use `.apply(funcao)`. A função recebe cada valor e devolve um novo. `.map(...)` é parecido, mas serve só para Series e aceita um dicionário para tradução direta.

```
cores = {"maxima": "vermelho", "media": "amarelo", "minima": "verde"}
df["cor"] = df["urgencia"].map(cores)
```

3.14. Recapitulando

- DataFrame é tabela; Series é coluna.
- Você lê com `read_csv/read_excel/read_json`.
- Inspecciona com `head`, `info`, `describe`, `value_counts`.
- Filtra com colchetes e condições booleanas (use `&` |).
- Agrupa com `groupby + agg`.
- Converte data com `pd.to_datetime` e usa o acessor `.dt`.
- Trata vazios com `isna/fillna/dropna`.
- Combina tabelas com `merge` (por chave) e `concat` (empilhar).

[PERGUNTA]

Antes de avançar para a Parte 4: pegue um .csv qualquer (pode ser fictício do Atende) e responda, com código, estas três perguntas: **(1)** Quantas solicitações em andamento existem? **(2)** Qual o setor com mais retornos pendentes? **(3)** Em quais dias da semana entram mais solicitações? Você já tem todas as ferramentas necessárias.

[LEITURA]

User Guide oficial Pandas: https://pandas.pydata.org/docs/user_guide/
Tutorial DataCamp (em inglês, com exercícios):
<https://www.datacamp.com/tutorial/pandas-read-csv>
Pandas em português (W3Schools traduz):
<https://www.w3schools.com/python/pandas/default.asp>

Parte 4 — Streamlit: do zero a primeira tela

Streamlit transforma um script Python em um site interativo, sem que voce precise aprender HTML, CSS ou JavaScript. E a ponte entre a logica de Pandas e a interface visual do dashboard.

4.1. O que e Streamlit

[CONCEITO]

Streamlit e uma biblioteca Python para criar interfaces web. Voce escreve Python normal, salva o arquivo, roda um comando — e abre uma pagina web no navegador. Cada vez que voce mexe em um botao ou campo, o script roda de novo do inicio com os novos valores. Pense numa *calculadora gigante*: voce digita os numeros, ela calcula tudo de novo e mostra o resultado.

[LEITURA]

Documentacao oficial (release 2026): <https://docs.streamlit.io/>
Notas da versao 2026 (novidades como `st.menu_button` e `filter_mode`):
<https://docs.streamlit.io/develop/quick-reference/release-notes/2026>

4.2. Instalando e rodando o primeiro app

```
pip install streamlit
```

Crie um arquivo `app.py` com este conteudo:

```
import streamlit as st

st.title("Portal Atende")
st.write("Bem-vindo ao dashboard.")
```

No terminal do VS Code, execute:

```
streamlit run app.py
```

Uma janela do navegador vai abrir automaticamente, geralmente em `http://localhost:8501`. Sempre que voce salvar o arquivo, o Streamlit detecta a mudanca e oferece recarregar.

[DICA]

Para fechar o servidor, volte ao terminal e aperte `Ctrl+C`. Para nao recarregar manualmente a cada salvar, no canto superior direito da pagina ative "Always rerun".

4.3. Componentes de texto

Funcao	Para que serve
<code>st.title("...")</code>	Titulo grande no topo da pagina
<code>st.header("...")</code>	Titulo de secao (um nivel abaixo)

<code>st.subheader("...")</code>	Titulo menor, para subsecoes
<code>st.write(...)</code>	O canivete suico: aceita texto, numero, dataframe, grafico...
<code>st.markdown("...")</code>	Permite negrito , <i>italico</i> , listas, links
<code>st.caption("...")</code>	Texto pequeno e cinza, ideal para legendas

[DICA]

Quando estiver em duvida sobre qual funcao usar, comece por `st.write`. Ele e a navalha suica do Streamlit: aceita praticamente qualquer coisa e tenta exibir do jeito certo.

4.4. Componentes de entrada (inputs)

Funcao	Para que serve
<code>st.text_input("Buscar...")</code>	Campo de texto curto
<code>st.text_area("Descricao")</code>	Campo de texto multilinhas
<code>st.number_input("Idade")</code>	Numero (com setas para subir/descer)
<code>st.date_input("Data")</code>	Calendario para escolher data
<code>st.selectbox("Urgencia", lista)</code>	Dropdown com escolha unica
<code>st.multiselect("Setores", lista)</code>	Dropdown que aceita varios valores
<code>st.checkbox("Aceito")</code>	Caixa de marcar/desmarcar
<code>st.radio("Periodo", opcoes)</code>	Botoes de radio (escolha unica)
<code>st.button("Aplicar filtros")</code>	Botao que retorna True quando clicado
<code>st.file_uploader("Upload")</code>	Caixa para o usuario enviar arquivo

[CONCEITO]

Cada componente **devolve** o valor que o usuario escolheu. Voce guarda em uma variavel e usa o resto do script. Exemplo: `urgencia = st.selectbox("Urgencia", ["Todas", "Maxima", "Media", "Minima"])`. A variavel `urgencia` passa a conter a opcao escolhida e voce pode usar para filtrar o DataFrame.

[DICA]

Na versao 2026, `st.selectbox` e `st.multiselect` ganharam o parametro `filter_mode`: o usuario digita para procurar uma opcao em listas grandes. Util quando o filtro tem muitas opcoes (lista de matriculas, por exemplo).

4.5. Exibindo dados

Funcao	Para que serve
<code>st.dataframe(df)</code>	Tabela interativa (ordena, filtra, seleciona)
<code>st.table(df)</code>	Tabela estatica (sem interatividade)

<code>st.metric("Total", 32)</code>	Cartao com valor grande, ideal para KPIs
<code>st.json(dicionario)</code>	Mostra um dicionario formatado em JSON
<code>st.image("logo.png")</code>	Exibe imagem

[DICA]

`st.dataframe` e quase sempre a melhor escolha para mostrar uma tabela. Ele permite que o usuario clique nos cabecalhos para ordenar, e na versao 2026 voce pode configurar selecao de linhas com `selection="single-row-required"`.

4.6. Layout: organizando a tela

Funcao	Para que serve
<code>st.set_page_config(layout="wide")</code>	Configura a pagina (chame uma vez no inicio)
<code>st.sidebar</code>	Painel lateral esquerdo (use com <code>'with st.sidebar:'</code> ou <code>st.sidebar.title()</code>)
<code>st.columns(3)</code>	Divide a area em 3 colunas lado a lado
<code>st.container(border=True)</code>	Caixa que agrupa varios componentes
<code>st.tabs(["Mes", "Semana"])</code>	Abas (como navegador)
<code>st.expander("Detalhes")</code>	Caixa que abre/fecha ao clicar

[CONCEITO]

`st.set_page_config` e **obrigatorio** ser a primeira chamada Streamlit do arquivo, antes de qualquer outro `st.qualquer_coisa`. E nele que voce define o titulo da aba do navegador, o layout largo ("wide") ou centralizado ("centered"), e o icone da pagina.

```
st.set_page_config(
    page_title="Atende UCB",
    layout="wide",
    initial_sidebar_state="expanded",
)
```

Para usar a sidebar, ha duas formas equivalentes:

```
st.sidebar.title("Menu")
st.sidebar.button("Calendario")

# ou, mais limpo:
with st.sidebar:
    st.title("Menu")
    st.button("Calendario")
```

[DICA]

`st.columns([2, 5, 2])` aceita uma lista de pesos. Aqui a coluna do meio fica 2,5x mais larga que as laterais. E como uma grade de CSS, mas em Python. Use isso para reproduzir layouts mais complexos das telas do Figma.

4.7. Notificacoes e mensagens

`st.info(...)` (azul, informativo), `st.success(...)` (verde, sucesso), `st.warning(...)` (amarelo, atencao), `st.error(...)` (vermelho, erro). Na versao 2026, se voce comecar a mensagem com o nome de um icone Material entre dois pontos (ex: `":warning: cuidado!"`), o Streamlit usa esse icone automaticamente.

4.8. Multi-paginas (uma pasta por tela)

[CONCEITO]

O Streamlit reconhece **automaticamente** qualquer arquivo `.py` colocado em uma pasta chamada `pages/` (no mesmo nivel do seu arquivo principal) e cria um item no menu lateral. E perfeito para o nosso projeto: uma pagina para o Calendario, outra para Solicitacoes, outra para Retornos.

```
meu_projeto/  
  app.py          # pagina de login  
  pages/  
    1_Calendario.py  
    2_Solicitacoes.py  
    3_Retornos.py
```

[DICA]

Os numeros no inicio do nome (`1_`, `2_`, `3_`) controlam a ordem que aparecem no menu, mas nao viram parte do titulo exibido. Os underscores viram espacos. `"1_Lista_de_Solicitacoes.py"` vira "Lista de Solicitacoes" no menu.

4.9. Estado entre cliques (session_state)

[CONCEITO]

Como o script roda do zero a cada interacao, voce precisa de um lugar para guardar coisas que devem sobreviver entre os cliques. Esse lugar e `st.session_state`, um dicionario especial. Funciona como uma agenda da sessao do usuario.

```
if "contador" not in st.session_state:  
    st.session_state.contador = 0  
  
if st.button("Incrementar"):  
    st.session_state.contador += 1  
  
st.write(st.session_state.contador)
```

[PERGUNTA]

Pesquise: como voce usaria `st.session_state` para guardar se o usuario fez login ou nao? Que valor voce salvaria? Em que momento o codigo verifica esse valor?

4.10. Cache: não recarregar a planilha 100 vezes

[CONCEITO]

Cada interação reroda o script. Se você lê um arquivo grande dentro do script, ele é lido toda vez — lento e desnecessário. O decorador `@st.cache_data` pede para o Streamlit guardar o resultado e só refazer se as entradas mudarem.

```
@st.cache_data
def carregar_solicitacoes():
    return pd.read_csv("solicitacoes.csv")

df = carregar_solicitacoes()
```

[DICA]

Regra prática: use `@st.cache_data` em qualquer função que carrega arquivo, faz consulta a banco ou chama API. Para dados que mudam (DataFrames filtrados pelo usuário, por exemplo), **não** coloque cache: você quer que rodem sempre.

4.11. Recapitulando

bulletStreamlit transforma script Python em página web; rode com `streamlit run app.py`.

bulletTexto: `title`, `header`, `write`, `markdown`.

bulletInputs: `text_input`, `selectbox`, `multiselect`, `date_input`, `button`.

bulletDados: `dataframe`, `table`, `metric`.

bulletLayout: `set_page_config`, `sidebar`, `columns`, `container`, `tabs`.

bulletMulti-páginas: pasta `pages/`.

bulletEstado: `st.session_state`; cache: `@st.cache_data`.

[PERGUNTA]

Antes de avançar para a Parte 5: monte um app de uma página só, com sidebar com 3 botões, uma área principal dividida em 2 colunas, um `st.metric` em cada coluna e uma tabela de exemplo embaixo. Use dados inventados na unha. Esse exercício é o seu kit básico — você vai reusar a estrutura nas 4 telas.

[LEITURA]

API Reference oficial: <https://docs.streamlit.io/develop/api-reference>

Layouts e containers: <https://docs.streamlit.io/develop/api-reference/layout>

Galeria de exemplos: <https://streamlit.io/gallery>

Parte 5 — Checklist por tela do Figma

Para cada uma das quatro telas já desenhadas, este capítulo apresenta os elementos a reproduzir, os componentes Streamlit sugeridos e perguntas-guia. **O código e por sua conta** — o roteiro não entrega resposta pronta.

[CONCEITO]

Como usar este capítulo: abra o Figma ao lado do roteiro e do VS Code. Para cada tela, identifique no Figma o componente que vai reproduzir, leia a sugestão do Streamlit (quase sempre você tem mais de uma opção!), e responda mentalmente as perguntas-guia antes de digitar a primeira linha. Se travar, volte as Partes 3 e 4.

5.1. Tela A — Login Portal-Atende

Tela A — Login Portal-Atende

*Fundo azul UCB ocupando a tela inteira. Logo da Universidade Católica centralizado.
Texto institucional. Botão branco com label "Portal - Atende".
(Consulte o arquivo Figma original para detalhes visuais finos.)*

Elementos a reproduzir

Elemento	Sugestão Streamlit
Fundo colorido inteiro	CSS injetado via <code>st.markdown(unsafe_allow_html=True)</code>
Logo centralizado	<code>st.image("logo_ucb.png", width=...)</code> dentro de <code>st.columns</code>
Título institucional	<code>st.markdown</code> com fonte branca e centralizada
Botão 'Portal - Atende'	<code>st.button("Portal - Atende")</code>
Centralização geral	<code>st.set_page_config(layout="centered")</code> + <code>st.columns([1,2,1])</code>

[DICA]

Para o fundo azul cobrir a tela, use CSS via `st.markdown` com `unsafe_allow_html=True`. Pesquise pelo seletor `.stApp`, que envolve a aplicação toda.

[PERGUNTA]

Como voce faria o botao "Portal - Atende" navegar para a tela de Calendario quando clicado? Pesquise `st.switch_page` ou o uso de `st.session_state` para controlar o login.

[PERGUNTA]

Como deixar o botao com largura fixa em vez de ocupar a coluna inteira? Que parametro do `st.button` controla isso? E se voce quiser largura total, qual usar?

5.2. Tela B — Calendario de Solicitacoes

Tela B — Calendario de Solicitacoes

Sidebar azul com itens Calendario / Solicitacoes / Retorno. Cabecalho com mes e seletor de ano. Controles < > Hoje, abas Mes/Semana/Dia, grade do calendario com eventos coloridos. Painel lateral direito com legenda das urgencias e setores. Botao "Upload de arquivos".

Elementos a reproduzir

Elemento	Sugestao Streamlit / extensao
Sidebar (menu lateral)	with st.sidebar: st.button(...) — uma chamada por item
Cabecalho com mes	st.title + st.markdown ou st.header
Seletor de mes/ano	st.selectbox ou st.date_input
Controles < > Hoje	Tres st.button() em st.columns([1,1,1,5])
Abas Mes/Semana/Dia	st.tabs(["Mes", "Semana", "Dia"])
Grade do calendario	Componente externo: streamlit-calendar (pip install)
Legenda colorida lateral	st.container(border=True) + lista de st.markdown
Upload de arquivos	st.file_uploader (aceita .csv, .xlsx)

[CONCEITO]

O Streamlit puro não tem um componente de calendário visual. A comunidade criou o **streamlit-calendar** (instale com `pip install streamlit-calendar`). Ele aceita uma lista de eventos no formato esperado pelo FullCalendar (título, data início, data fim, cor) — exatamente o que você extrai do DataFrame de solicitações.

[LEITURA]

Documentação streamlit-calendar (componente externo):
<https://github.com/im-perativa/streamlit-calendar>

[PERGUNTA]

Como você transformaria seu DataFrame de solicitações na lista de eventos que o streamlit-calendar espera? Que colunas precisam virar "title", "start" e "backgroundColor"? (Dica: revise `apply` e `to_dict("records")` da Parte 3.)

[PERGUNTA]

O que acontece se a coluna de data estiver como string em vez de datetime? O calendário consegue ler? Pesquise o formato ISO 8601 que o FullCalendar espera.

5.3. Tela C — Lista de Solicitações

Tela C — Lista de Solicitações

*Cabeçalho com título e total entre parênteses (ex: '32'). Botão + Nova Solicitação.
Linha de filtros: campo de busca, selects Urgência/Status/Ordenar.
Cards verticais com bullet colorido + título + descrição + data + autor + badges.
Paginação no rodapé. Painel de filtros avançado à direita (período, urgência, status, botoes).*

Elementos a reproduzir

Elemento	Sugestão Streamlit
Total entre parênteses	f-string com <code>len(df_filtrado)</code> dentro de <code>st.markdown</code>
Botão + Nova Solicitação	<code>st.button(":material/add: Nova Solicitação")</code>
Campo de busca	<code>st.text_input("", placeholder="Buscar solicitações...")</code>

Selects Urgencia/Status/Ordenar	tres <code>st.selectbox</code> em <code>st.columns(3)</code>
Cards de solicitacao	<code>for linha in df.itertuples():</code> with <code>st.container(border=True):</code> ...
Bullet colorido	<code>st.markdown</code> com HTML inline (<code>span</code> com <code>background-color</code>)
Badges (Urgencia/Status)	<code>st.markdown</code> com background colorido por urgencia
Paginacao	<code>st.session_state</code> + slice no <code>DataFrame</code> ou <code>st.pagination</code> (versao 2026)
Painel de filtros direito	Coluna direita criada com <code>st.columns([3,1])</code>
Periodo (calendario)	<code>st.date_input(value=(inicio, fim))</code>
Radios urgencia	<code>st.radio</code> com options coloridas
Botoes Limpar / Aplicar	Dois <code>st.button</code> no fim do painel

[CONCEITO]

Cards sao apenas containers com borda. Cada solicitacao no `DataFrame` vira um `st.container(border=True)` dentro de um loop `for`. Dentro do container voce coloca `st.columns` para alinhar bullet, texto e badge — exatamente como uma linha do Figma.

[DICA]

`st.container(border=True)` e novo (Streamlit 2024+). Se quiser mais controle visual, voce pode injetar CSS com `st.markdown`. Mas tente começar simples: o padrao do Streamlit ja parece um dashboard.

[PERGUNTA]

Como voce filtra o `DataFrame` conforme o usuario muda os selects? (Lembre: o script reroda do zero a cada interacao. Voce le os selects, monta a condicao de filtro, aplica, e ai itera para mostrar os cards.)

[PERGUNTA]

Quando o usuario clica em "Limpar filtros", como voce reseta todos os campos? Pesquise se da para mexer em `st.session_state[chave]` a partir de um botao.

[PERGUNTA]

Como destacar 5 cards por pagina? Que valor voce guardaria em `st.session_state`? Como calcular o slice `df.iloc[inicio:fim]`?

5.4. Tela D — Lista de Retornos

Tela D — Lista de Retornos

*Estrutura praticamente identica a Tela C, mas para retornos enviados pelos setores.
Filtros principais: Setor (Academico/Administrativo/Financeiro/Todos) e Status (Lido/Nao Lido/Favorito/Spam).
Cards mostram setor (com cor), titulo do retorno, texto curto, data, autor e estado de leitura.*

Elementos novos / diferentes da Tela C

Elemento	Sugestao Streamlit
Bullet com cor por setor	Mesma logica da urgencia, mas mapeando setor -> cor
Badge "Setor Academico/etc."	st.markdown com background dependendo do valor
Status Lido/Nao Lido/Favorito/Spam	st.radio com 5 opcoes (incluindo "Todos")
Reuso do componente "Card"	Extraia o card de Solicitacao em uma funcao Python

[CONCEITO]

Quando voce percebe que duas telas tem o mesmo padrao de card, e hora de extrair uma **funcao**. Por exemplo, `def card_solicitacao(linha): ...` recebe uma linha do DataFrame e desenha o card. Voce chama essa funcao no for de ambas as telas — DRY (Don't Repeat Yourself, nao se repita).

[PERGUNTA]

Como sua estrutura de pastas vai ficar? Onde vai ficar a funcao `card_solicitacao` para que tanto a Tela C quanto a Tela D consigam importa-la? Pesquise sobre arquivos `utils.py` ou `components.py`.

[PERGUNTA]

A Tela C usa solicitacoes; a Tela D usa retornos. As colunas dos DataFrames sao diferentes. Como adaptar a funcao `card_solicitacao` para servir aos dois? Use **dicionarios** ou **parametros opcionais**. Qual estrategia parece mais limpa?

5.5. Telas adicionais (a chegar)

O usuario do roteiro mencionou que ha mais 2 telas em desenvolvimento. Quando elas chegarem, use o mesmo modelo: **placeholder visual + tabela de elementos + dicas + perguntas-guia**. Nao

tente reaproveitar componentes de outras telas sem antes desenhar a sua mentalmente.

[DICA]

Sempre **desenhe primeiro no Figma**, depois reproduza no Streamlit. Tentar improvisar uma tela direto no código costuma terminar em código bagunçado e UI inconsistente.

5.6. Recapitulando a Parte 5

- bullet Cada tela do Figma vira um arquivo `.py` em `pages/`.
- bullet `Cards = st.container(border=True)` dentro de um `for`.
- bullet Componentes externos (`streamlit-calendar`) cobrem o que o Streamlit puro não tem.
- bullet Quando duas telas se parecem, extraia funções para reutilizar.
- bullet Os filtros são apenas inputs Streamlit que produzem variáveis para alimentar o filtro do `DataFrame`.

[PERGUNTA]

Antes de avançar para a Parte 6: monte um esqueleto vazio do projeto, com um arquivo `app.py` e quatro arquivos em `pages/`, cada um só com `st.title` e o nome da tela. Rode `streamlit run app.py` e veja o menu aparecer. Já é meio caminho andado!

Parte 6 — Pandas + Streamlit juntos

As partes anteriores ensinaram cada peça isoladamente. Aqui você monta o quebra-cabeça: Pandas processa dados, Streamlit mostra. A integração bem feita é o que diferencia um dashboard amador de um profissional.

6.1. O fluxo padrão de um dashboard

[CONCEITO]

Carregar -> filtrar -> exibir. Esta é a sequência que você vai repetir em cada página. Você carrega o DataFrame original (uma vez, com cache), o usuário interage com filtros, você aplica os filtros e mostra o resultado. Quando o usuário muda algo, o Streamlit reroda tudo do início com os novos valores.

```
@st.cache_data
def carregar():
    return pd.read_csv("solicitacoes.csv")

df = carregar()

urgencia = st.selectbox("Urgencia", ["Todas"] + list(df["urgencia"].unique()))

if urgencia != "Todas":
    df = df[df["urgencia"] == urgencia]

st.dataframe(df)
```

6.2. Metricas no topo (KPIs)

Os números grandes que você vê em cima de dashboards profissionais são chamados de KPIs (Key Performance Indicators). No Streamlit você os faz com `st.metric`, geralmente dentro de `st.columns` para alinhar vários lado a lado.

```
col1, col2, col3 = st.columns(3)
col1.metric("Total", len(df))
col2.metric("Urgencia maxima", (df["urgencia"] == "maxima").sum())
col3.metric("Pendentes", (df["status"] == "pendente").sum())
```

[DICA]

`st.metric` tem um terceiro parâmetro `delta` que mostra uma seta verde/vermelha para indicar variação. Útil para comparar com o mês anterior.

6.3. Filtros que conversam entre si

[CONCEITO]

Quando voce tem varios filtros, eles devem ser **combinados**: o usuario escolhe urgencia e status e intervalo de data, e o resultado e a intersecao. A logica e: comeca com o DataFrame inteiro e vai aplicando uma condicao de cada vez, sempre sobrescrevendo a variavel.

```
df_filtrado = df.copy()
if urgencia != "Todas":
    df_filtrado = df_filtrado[df_filtrado["urgencia"] == urgencia]
if status != "Todos":
    df_filtrado = df_filtrado[df_filtrado["status"] == status]
df_filtrado = df_filtrado[
    (df_filtrado["data"] >= data_inicio) & (df_filtrado["data"] <= data_fim)
]
```

[DICA]

Sempre faca `df.copy()` antes de comecar a filtrar. Senao, voce pode acabar modificando o DataFrame original sem querer (e o Streamlit cacheia ele!) — isso causa bugs estranhos onde os dados "desaparecem" apos varios cliques.

6.4. Boas praticas de organizacao

- **Separe dados de exibicao.** Coloque a logica Pandas em uma funcao (`filtrar_solicitacoes(df, ...)`) e a apresentacao Streamlit fora.
- **Um arquivo, uma responsabilidade.** Pagina `pages/2_Solicitacoes.py` nao deveria saber detalhes do Calendario.
- **Crie um modulo** `utils.py` com funcoes que se repetem (carregar dados, formatar data, mapear urgencia para cor).
- **Cache so em funcoes de carregamento.** Filtragem e leve, nao precisa.
- **Use nomes claros.** `df` esta ok, mas `df_solicitacoes_filtradas` e melhor quando coexistem varios.

[PERGUNTA]

Como voce organizaria o seu projeto em pastas? Reproduza no quadro: o que vai em `app.py`, em `pages/`, em `utils.py`, em `data/` (planilhas brutas)? Lembre que arquivos pequenos sao mais faceis de manter.

6.5. Recapitulando

- Fluxo: carregar (com cache) -> filtrar (sem cache) -> exibir.
- KPIs com `st.metric` em colunas.
- Sempre use `df.copy()` antes de filtrar para nao alterar o original.
- Funcoes em `utils.py` reduzem repeticao.
- Voce nao precisa fazer nada "avancado". Combinar bem o basico ja faz um dashboard solido.

[PERGUNTA]

Implemente, ainda sem se preocupar com aparência, um dashboard funcional que: lê uma planilha, mostra tres KPIs no topo, oferece tres filtros (urgencia, status, periodo) e apresenta a tabela filtrada. Quando isso funcionar ponta a ponta, voce ja tem 60% do projeto.

Parte 7 — Extra: SQL no Pandas com DuckDB

Esta parte é **opcional**. Você já consegue construir o dashboard inteiro só com Pandas. DuckDB é uma alternativa moderna para quem prefere SQL — e é ótima para impressionar na apresentação final.

7.1. Por que aprender SQL

[CONCEITO]

SQL (Structured Query Language) é a linguagem usada para consultar bancos de dados há mais de 50 anos. Saber SQL é quase obrigatório para qualquer profissional de tecnologia, porque a maioria dos sistemas guarda dados em bancos relacionais (Postgres, MySQL, SQL Server, etc.). Aprender SQL ao mesmo tempo que Pandas **amplia o seu kit de ferramentas**.

Pandas e SQL fazem coisas parecidas com sintaxes diferentes. Algumas operações ficam mais elegantes em SQL (juntar várias tabelas com critérios complexos, por exemplo); outras ficam melhores em Pandas (transformações ponto a ponto). Ter as duas é o ideal.

7.2. O que é DuckDB

[CONCEITO]

DuckDB é um banco de dados "em processo": ele roda dentro do seu código Python, sem servidor para configurar. Você instala com `pip install duckdb` e já pode rodar SQL contra DataFrames como se fossem tabelas. É rápido, moderno e é o padrão atual da indústria para análises locais (em 2026, substituí o antigo `pandasql`).

[LEITURA]

Doc oficial DuckDB sobre Pandas:

https://duckdb.org/docs/current/guides/python/sql_on_pandas

Tutorial MotherDuck (escrito pelos criadores):

<https://motherduck.com/learn/duckdb-python-quickstart-part1/>

7.3. Instalação e primeira consulta

```
pip install duckdb

import duckdb
import pandas as pd

df = pd.read_csv("solicitacoes.csv")

resultado = duckdb.sql("SELECT * FROM df WHERE urgencia = 'maxima'").df()
print(resultado)
```

Repare na magia: voce escreve `FROM df`, e o DuckDB descobre que `df` e a variavel Python contendo o DataFrame. O `.df()` no final converte o resultado de volta para um DataFrame Pandas, que voce pode usar normalmente no Streamlit.

7.4. Sintaxe SQL essencial

Operacao	Pandas	SQL (DuckDB)
Selecionar colunas	<code>df[["a", "b"]]</code>	<code>SELECT a, b FROM df</code>
Filtrar	<code>df[df.x == 1]</code>	<code>SELECT * FROM df WHERE x = 1</code>
Ordenar	<code>df.sort_values("x")</code>	<code>SELECT * FROM df ORDER BY x</code>
Agrupar e contar	<code>df.groupby("x").size()</code>	<code>SELECT x, COUNT(*) FROM df GROUP BY x</code>
Juntar tabelas	<code>df1.merge(df2, on="id")</code>	<code>SELECT * FROM df1 JOIN df2 USING(id)</code>
Limitar resultado	<code>df.head(10)</code>	<code>SELECT * FROM df LIMIT 10</code>

[DICA]

Note que SQL usa `=` para comparar (em Python e `==`) e aceita aspas simples ou duplas para texto. Outra diferenca: SQL e **case-insensitive** para palavras-chave (`SELECT`, `select`, `Select` sao iguais), mas **case-sensitive** para nomes de coluna em alguns dialetos.

7.5. Exemplo no contexto do Atende

Imagine que voce queira responder: *"quantas solicitacoes de cada urgencia existem em cada setor?"*. Em SQL com DuckDB:

```
consulta = """
    SELECT setor, urgencia, COUNT(*) AS total
    FROM df
    GROUP BY setor, urgencia
    ORDER BY total DESC
    """

resumo = duckdb.sql(consulta).df()
st.dataframe(resumo)
```

O DataFrame resultante pode ir direto para um `st.dataframe` ou ate para um `st.bar_chart`. SQL e Streamlit conversam pelo Pandas.

7.6. Quando preferir SQL ou Pandas

- **Prefira SQL** para perguntas que envolvem juntar varias tabelas, agrupar com varias condicoes, ou ordenar/limitar de forma sofisticada.
- **Prefira Pandas** para transformacoes ponto a ponto (aplicar uma funcao em cada linha) e quando voce ja esta no meio de uma cadeia de operacoes.
- **Voce pode misturar:** faz a parte pesada em SQL com DuckDB, devolve para Pandas com `.df()`, e finaliza com transformacoes Pandas.

[PERGUNTA]

Tente reescrever, em SQL, os tres exercicios da Parte 3.14 ("quantas em andamento", "setor com mais retornos pendentes", "dias da semana com mais solicitacoes"). Compare com a versao Pandas que voce ja fez. Qual ficou mais legivel para voce?

[PERGUNTA]

Pesquise: o que e uma **CTE** (Common Table Expression) e como ela ajuda a quebrar uma consulta grande em pedacos? Veja se DuckDB suporta. Esse e um conceito que profissionais SQL usam o tempo todo.

7.7. Recapitulando

- DuckDB roda SQL contra DataFrames sem servidor.
- `duckdb.sql("...").df()` e o caminho de ida e volta.
- SQL essencial: SELECT, WHERE, ORDER BY, GROUP BY, JOIN, LIMIT.
- SQL e Pandas se complementam — escolha o melhor para cada operacao.

[LEITURA]

Comparacao Pandas <-> SQL (oficial):
https://pandas.pydata.org/docs/getting_started/comparison/comparison_with_sql.html
Tutorial em portugues sobre SQL basico: <https://www.w3schools.com/sql/>
DuckDB Quickstart: <https://duckdb.org/docs/api/python/overview>

Parte 8 — Recursos, glossario e proximos passos

Programar e um maraton, nao uma corrida. Esta secao reúne as fontes confiaveis para voce consultar sempre que travar — e ele vai travar.

8.1. Como ler mensagens de erro

[CONCEITO]

Toda vez que o Python encontra um problema, ele imprime um **traceback** — um relato do que estava acontecendo quando o erro surgiu. Leia **de baixo para cima**:

Ultima linha: tipo do erro (NameError, TypeError, KeyError, IndentationError...) e descricao curta.

Linhas do meio: chamadas que levaram ate o erro, com numeros de linha — abra o arquivo e veja.

Primeira linha do bloco: contexto inicial (geralmente o seu arquivo principal).

[DICA]

Copie a mensagem completa e cole no Google. Inclua o nome da biblioteca ("pandas KeyError" encontra coisas mais especificas que so "KeyError").

Erros mais comuns no inicio:

Erro	O que costuma significar
NameError: name 'x' is not defined	Voce usou uma variavel/funcao antes de criar (ou errou o nome).
IndentationError	Espacos errados no inicio de uma linha.
KeyError: 'coluna'	Voce pediu uma coluna que nao existe no DataFrame.
TypeError: ... unsupported operand	Misturou tipos diferentes (str + int, por ex.).
FileNotFoundError	Caminho do arquivo errado ou arquivo nao salvo.
ValueError	Voce passou um valor invalido para uma funcao.
ModuleNotFoundError	Esqueceu de fazer pip install.

8.2. Onde pedir ajuda

Stack Overflow (<https://stackoverflow.com/>) — para qualquer duvida tecnica. Procure antes de perguntar; quase sempre alguem ja perguntou.

Streamlit Community (<https://discuss.streamlit.io/>) — forum oficial, otimo para componentes especificos.

r/learnpython (<https://www.reddit.com/r/learnpython/>) — comunidade acolhedora para iniciantes.

bullet ChatGPT, Claude, Gemini — colem a mensagem de erro e o trecho do seu código. Sempre revise o que a IA sugere antes de aceitar.

bullet YouTube — Curso em Vídeo, Programação Dinâmica, Hashtag Treinamentos (em português).

8.3. Documentações oficiais (favorite todas)

bullet Python (PT-BR): <https://docs.python.org/pt-br/3/>

bullet Pandas (User Guide): https://pandas.pydata.org/docs/user_guide/

bullet Pandas (10 minutes to pandas): https://pandas.pydata.org/docs/user_guide/10min.html

bullet Pandas (comparação com SQL):
https://pandas.pydata.org/docs/getting_started/comparison/comparison_with_sql.html

bullet Streamlit (docs): <https://docs.streamlit.io/>

bullet Streamlit (release notes 2026):
<https://docs.streamlit.io/develop/quick-reference/release-notes/2026>

bullet Streamlit (galeria): <https://streamlit.io/gallery>

bullet DuckDB (docs): <https://duckdb.org/docs/>

8.4. Glossário rápido

Termo	Significado curto
DataFrame	Tabela do Pandas (linhas e colunas com rótulos).
Series	Uma coluna isolada de um DataFrame.
Widget	Componente interativo do Streamlit (botão, input, select).
Callback	Função executada como reação a um evento (clique, mudança).
Cache	Memória temporária para evitar refazer trabalho pesado.
Sidebar	Painel lateral esquerdo da página Streamlit.
KPI	Key Performance Indicator — métrica importante exibida no topo.
JOIN	Operação SQL que une duas tabelas por uma coluna em comum.
GROUP BY	Agrupar linhas para aplicar funções (count, sum, avg).
NaN	"Not a Number" — célula vazia em Pandas.
Pip	Gerenciador de pacotes do Python.
IDE	Integrated Development Environment — o VS Code, no nosso caso.
Traceback	Relatório detalhado de um erro em Python.
Endpoint	Endereço onde uma API recebe requisições.

8.5. Projeto-bonus para fixar

Quando o dashboard estiver entregue, tente este desafio extra:

[PERGUNTA]

Adicione uma **nova pagina** chamada "Estatisticas" que mostre um grafico de barras com a contagem de solicitacoes por dia da semana, e um grafico de pizza com a distribuicao por urgencia. Use `st.bar_chart` e `st.plotly_chart`. Pesquise: o que o Plotly oferece a mais que o grafico padrao do Streamlit?

[PERGUNTA]

Como voce **publicaria** o seu dashboard para que qualquer um na UCB possa acessar? Pesquise **Streamlit Community Cloud** (gratuito para projetos publicos), **Render.com**, ou hospedagem em uma maquina interna da universidade. Cada caminho tem prosas e contras.

8.6. Mensagem final

Voce comecou este roteiro sem saber o que era uma variavel. Se chegou ate aqui — entendeu Pandas, conseguiu rodar Streamlit, reproduziu telas do Figma e talvez tenha experimentado SQL com DuckDB — voce ja esta fazendo coisas que muitos profissionais demoraram meses para aprender. **Programar nao e talento; e persistencia + pratica deliberada.** Continue construindo, continue quebrando, continue consertando. O Atende vai agradecer.

Boa sorte, e bom codigo.